

Contents	4. Math
1. Data Structure	4 - 1. Seive of Eratosthenes 4 - 2. Euler Phi 4 - 3. Modular Inverse 4 - 4. Extended Euclid 4 - 5. Chinese Remainder Theorm 4 - 6. Catalan Number 4 - 7. Binomial Coefficient (N^2) 4 - 8. Lucas Theorm (Binomial Coefficient)
2. Geometry	5. String
2 - 1. CCW 2 - 2. 360 degree sort 2 - 3. Convex Hull 2 - 4. Rotating Calipers 2 - 5. Point in Convex Polygon	5 - 1. Rabin-Karp 5 - 2. Trie 5 - 3. KMP 5 - 4. Manacher 5 - 5. Suffix Array
3. Graph	6. ETC
3 - 1. Bellman-Ford 3 - 2. Eulerian Circuit 3 - 3. Bipartitie Matching 3 - 4. Edmonds-Karp 3 - 5. Dinic 3 - 6. Minimum Cost Maximum Flow 3 - 7. Centroid Decomposition	6 - 1. Union Find 6 - 2. Ternary Search 6 - 3. Power with Divide and Conquer 6 - 4. Fibonacci Matrix 6 - 5. Convex Hull Trick
	7. Appendix
	7 - 1. Bit twiddling 7 - 2. etc 7 - 3. where to use

```
#include <bits/stdc++.h>
```

```
#define ll long long int
```

```
using namespace std;
```

```
/*
```

```
Segment Tree
```

```
*/
```

```
struct ST
```

```
{
```

```
    ll n;
```

```
    vector<ll> tree;
```

```
    ST(ll n) :n(n), tree(4 * (n + 1)) {}
```

```
    ll query(ll x, ll s, ll e, ll l, ll r)
```

```
    {
```

```
        if (l > e || r < s) return 0ll;
```

```
        if (l <= s && e <= r) return tree[x];
```

```
        ll L = query(x * 2, s, (s + e) / 2, l, r);
```

```
        ll R = query(x * 2 + 1, (s + e) / 2 + 1, e, l, r);
```

```
        return L + R;
```

```
    }
```

```

void update(ll x, ll s, ll e, ll idx, ll val)
{
    if (idx < s || idx > e) return;
    if (s == e) tree[x] = val;
    else
    {
        update(x * 2, s, (s + e) / 2, idx, val);
        update(x * 2 + 1, (s + e) / 2 + 1, e, idx, val);
        tree[x] = tree[x * 2] + tree[x * 2 + 1];
    }
}

};

/*
Lazy Propagation
*/

struct STLazy
{
    ll n;
    vector<ll> tree, lazy;
    STLazy(ll n) : n(n), tree(4 * (n + 1)), lazy(4 * (n + 1)) {}
    void update_lazy(ll x, ll s, ll e)
    {
        tree[x] += (e - s + 1) * lazy[x];
        if (s != e)
        {
            lazy[x * 2] += lazy[x];
            lazy[x * 2 + 1] += lazy[x];
        }
        lazy[x] = 0;
    }
    void update_range(ll x, ll s, ll e, ll l, ll r, ll diff)
    {
        update_lazy(x, s, e);
        if (l > e || s > r) return;
        if (l <= s && e <= r)
        {
            tree[x] += (e - s + 1) * diff;
            if (s != e)
            {
                lazy[x * 2] += diff;
                lazy[x * 2 + 1] += diff;
            }
        }
        else
        {
            update_range(x * 2, s, (s + e) / 2, l, r, diff);
            update_range(x * 2 + 1, (s + e) / 2 + 1, e, l, r, diff);
            tree[x] = tree[x * 2] + tree[x * 2 + 1];
        }
    }
    ll query(ll x, ll s, ll e, ll l, ll r)
    {
        update_lazy(x, s, e);
        if (l > e || r < s) return 0ll;
        if (l <= s && e <= r) return tree[x];
        ll L = query(x * 2, s, (s + e) / 2, l, r);
        ll R = query(x * 2 + 1, (s + e) / 2 + 1, e, l, r);
        return L + R;
    }
}

```

```

    }
};

/*
Kosaraju (SCC)
*/
#define MAX_V 100005
struct Kosaraju
{
    ll V, E, num = 0; // SCC number is 1-base
    vector<ll> g[MAX_V], rg[MAX_V], order, SCC; // node number is 1-base
    vector<bool> v;
    Kosaraju(ll V, ll E) :V(V), E(E), SCC(V + 1, 0), v(V + 1, 0) {}
    void dfs(ll x)
    {
        v[x] = 1;
        for (auto y : g[x]) if (!v[y]) dfs(y);
        order.push_back(x);
    }
    void rdfs(ll x, ll num)
    {
        v[x] = 1;
        SCC[x] = num;
        for (auto y : rg[x]) if (!v[y]) rdfs(y, num);
    }
    void go()
    {
        for (int i = 1; i <= V; i++) if (!v[i]) dfs(i);
        fill(v.begin(), v.end(), 0);
        reverse(order.begin(), order.end());
        for (auto i : order) if (!SCC[i]) rdfs(i, ++num);
    }
};

// Don't forget to build g[MAX_V] and rg[MAX_V]

```

```

/*
LCA
*/
#define log2MAX_V 22
struct LCA
{
    ll V, E;
    ll p[log2MAX_V][MAX_V];
    vector<ll> d, g[MAX_V];
    LCA(ll V, ll E) :V(V), E(E), d(V + 1) {}
    void dfs(ll x, ll par)
    {
        for (auto y : g[x])
            if (y != par)
            {
                p[0][y] = x;
                d[y] = d[x] + 1;
                dfs(y, x);
            }
    }
    void make(ll root)
    {
        dfs(root, 0);
        for (int i = 1; i < log2(V) + 1; i++)

```

```

        for (int j = 1; j <= V; j++)
            p[i][j] = p[i - 1][p[i - 1][j]];
    }
ll lca(ll u, ll v)
{
    if (d[u] < d[v]) swap(u, v);
    ll diff = d[u] - d[v];
    for (int i = 0; i < log2(V) + 1; i++)
        if (diff & (1ll << i)) u = p[i][u];
    if (u == v) return u;
    for (int i = log2(V); i >= 0; i--)
        if (p[i][u] != p[i][v]) u = p[i][u], v = p[i][v];
    return p[0][u];
}
};

/*
Merge Sort Tree
*/
#define MAX_N 100005
struct MergeSortTree
{
    vector<ll> a;
    vector<ll> tree[4 * MAX_N];
    void init(ll x, ll s, ll e)
    {
        if (s == e) tree[x].push_back(a[s]);
        else
        {
            init(x * 2, s, (s + e) / 2);
            init(x * 2 + 1, (s + e) / 2 + 1, e);
            auto& l = tree[x * 2], & r = tree[x * 2 + 1];
            tree[x].resize(l.size() + r.size());
            merge(l.begin(), l.end(), r.begin(), r.end(), tree[x].begin());
        }
    }
    ll query(ll x, ll s, ll e, ll l, ll r, ll v)
    {
        if (r < s || e < l) return 0;
        if (l <= s && e <= r)
            return lower_bound(tree[x].begin(), tree[x].end(), v) - tree[x].begin();
        ll L = query(x * 2, s, (s + e) / 2, l, r, v);
        ll R = query(x * 2 + 1, (s + e) / 2 + 1, e, l, r, v);
        return L + R; // index of v
    }
};

/*
Fenwick Tree
*/
struct FenwickTree {
    int n;
    vector<long long> tree;

    FenwickTree(int n) : n(n), tree(n + 1, 0) {}

    void update(int idx, long long delta) {
        while (idx <= n) {

```

```

        tree[idx] += delta;
        idx += (idx & -idx);
    }
}

long long query(int idx) {
    long long sum = 0;
    while (idx > 0) {
        sum += tree[idx];
        idx -= (idx & -idx);
    }
    return sum;
}

long long range_query(int left, int right) {
    if (left > right) return 0;
    return query(right) - query(left - 1);
}
};

/*
Sparse Table
*/
class SparseTable {
private:
    int n;
    int K;
    vector<vector<int>> st;
    vector<int> lg;

public:
    SparseTable(const vector<int>& arr) {
        n = arr.size();
        K = __lg(n) + 1;

        st.assign(n, vector<int>(K));
        lg.assign(n + 1, 0);

        for (int i = 2; i <= n; i++) {
            lg[i] = lg[i / 2] + 1;
        }

        for (int i = 0; i < n; i++) {
            st[i][0] = arr[i];
        }

        for (int j = 1; j < K; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
            }
        }
    }

    int query(int L, int R) {
        int j = lg[R - L + 1];
        return min(st[L][j], st[R - (1 << j) + 1][j]);
    }
};

```

```

/*
CCW
*/
ll ccw(pair<ll, ll> A, pair<ll, ll> B, pair<ll, ll> C)
{
    return A.first * B.second + B.first * C.second + C.first * A.second - A.first * C.second - B.first * A.second -
    C.first * B.second;
}

/*
360 degree sort
*/
pair<ll, ll> operator - (const pair<ll, ll> a, const pair<ll, ll> b) { return { a.first - b.first, a.second -
b.second }; }
pair<ll, ll> F;
bool cmp(pair<ll, ll> a, pair<ll, ll> b)
{
    a = a - F; b = b - F;
    if ((a.first < 0) != (b.first < 0)) return a.first > b.first;
    if (a.first == 0 && b.first == 0) return a.second < b.second;
    return a.second * b.first < b.second * a.first;
}

/*
Convex Hull
*/
struct ConvexHull
{
    vector<pair<ll, ll>> v, L, R;
    ConvexHull(ll N) :v(N) {}
    void TwoHalf()
    {
        sort(v.begin(), v.end());
        for (auto i : v)
        {
            while (L.size() >= 2 && ccw(L[L.size() - 2], L[L.size() - 1], i) <= 0) L.pop_back();
            L.push_back(i);
            while (R.size() >= 2 && ccw(R[R.size() - 2], R[R.size() - 1], i) >= 0) R.pop_back();
            R.push_back(i);
        }
        v.clear();
        for (ll i = 0; i < L.size() - 1; i++) v.push_back(L[i]);
        for (ll i = R.size() - 1; i > 0; i--) v.push_back(R[i]);
    }
};

/*
Rotating Calipers
*/
#define pll pair<ll, ll>
pll operator + (const pll a, const pll b) { return { a.first + b.first, a.second + b.second }; }
pll operator - (const pll a, const pll b) { return { a.first - b.first, a.second - b.second }; }
struct RC
{
    ll dist(const pll a, const pll b)
    {
        return (a.first - b.first) * (a.first - b.first) + (a.second - b.second) * (a.second - b.second);
    }
}

```

```

ll ccw(const pll a, const pll b, const pll c)
{
    return a.first * b.second + b.first * c.second + c.first * a.second - a.first * c.second - b.first * a.second -
c.first * b.second;
}

ll N;
vector<pair<ll, ll>> v, L, R;
RC(ll N) :N(N), v(N) {}
void TwoHalf()
{
    sort(v.begin(), v.end());
    for (auto i : v)
    {
        while (L.size() >= 2 && ccw(L[L.size() - 2], L[L.size() - 1], i) <= 0) L.pop_back();
        L.push_back(i);
        while (R.size() >= 2 && ccw(R[R.size() - 2], R[R.size() - 1], i) >= 0) R.pop_back();
        R.push_back(i);
    }
    v.clear();
    for (ll i = 0; i < L.size() - 1; i++) v.push_back(L[i]);
    for (ll i = R.size() - 1; i > 0; i--) v.push_back(R[i]);
}

ll go() // v should be convex hull
{
    ll ret = 0;
    for (int i = 0, r = 0; i < N; i++)
    {
        // A, B, D - C + B
        while (r < N * 2 && ccw(v[i], v[(i + 1) % N], v[(i + 1) % N] + v[(r + 1) % N] - v[r % N]) >= 0)
        {
            ret = max(ret, dist(v[i], v[r % N])); // A, D
            r++;
        }
        ret = max(ret, dist(v[i], v[r % N]));
    }
    return ret;
}

};

/*
Point in Convex Polygon
*/

bool isInCP(vector<pair<int, int>>& v, pair<int, int> pt) // v which is convex hull is sorted based on counter clock
wise
{
    int vs = v.size();
    if (vs == 0) return false;
    if (vs == 1) return v[0] == pt;
    if (vs == 2) return ccw(v[0], v[1], pt) == 0 && v[0] <= pt && pt <= v[1];
    if (ccw(v[vs - 1], v[0], pt) < 0 || ccw(v[0], v[1], pt) < 0) return 0;
    int lo = 1, hi = vs - 1, res = vs - 1;
    while (lo <= hi)
    {
        int mid = (lo + hi) / 2;
        if (ccw(v[0], v[mid], pt) >= 0) lo = mid + 1;
        else
        {
            res = mid;
            hi = mid - 1;
        }
    }
}

```

```

    }
}

return ccw(v[res - 1], v[res], pt) >= 0;
}

/*
Bellman-Ford
*/
#define MAX_V 100005
#define INF 1e9
struct BellmanFord
{
    ll V;
    vector<pair<ll, ll>> g[MAX_V];
    BellmanFord(ll V) : V(V) {}
    vector<ll> bellmanFord(ll src) // if there is a negative cycle, returns empty vector.
    {
        vector<ll> upper(V, INF);
        upper[src] = 0;
        bool updated = 0;
        for (int iter = 0; iter < V; iter++)
        {
            updated = 0;
            for (int here = 0; here < V; here++)
                for (auto i : g[here])
                {
                    auto [there, cost] = i;
                    if (upper[there] > upper[here] + cost)
                    {
                        upper[there] = upper[here] + cost;
                        updated = 1;
                    }
                }
            if (!updated) break;
        }
        if (updated) upper.clear(); // there is negative cycle.
        return upper;
    }
};

/*
Eulerian Circuit
*/
#define MAX_V 1005
struct Eulerian
{
    ll V, start;
    ll g[MAX_V][MAX_V]; // adjacency matrix
    vector<ll> res;
    bool check()
    {
        for (int i = 1; i <= V; i++)
        {
            ll cnt = 0;
            for (int j = 1; j <= V; j++) cnt += g[i][j];
            if (cnt & 2) return 0;
            if (!start && cnt) start = i;
        }
    }
}

```



```

void dfs(ll x)
{
    for (int y = 1; y <= V; y++)
    {
        if (g[x][y])
        {
            g[x][y]--;
            g[y][x]--;
            dfs(y);
        }
    }
    res.push_back(x);
}

};

/*
Bipartite Matching
*/
#define MAX_V 1005
struct Bipartite
{
    ll V, res = 0;
    vector<ll> g[MAX_V], A, B;
    vector<bool> v;
    Bipartite(ll V) : V(V), A(V + 1, -1), B(V + 1, -1), v(V + 1, 0) {};
    bool dfs(ll a)
    {
        v[a] = 1;
        for (auto b : g[a])
            if (B[b] == -1 || (!v[B[b]] && dfs(B[b])))
            {
                A[a] = b;
                B[b] = a;
                return 1;
            }
        return 0;
    }
    void go()
    {
        for (int i = 1; i <= V; i++)
            if (A[i] == -1)
            {
                fill(v.begin(), v.end(), 0);
                if (dfs(i)) res++;
            }
    }
};

/*
Edmonds-Karp
*/
struct EdmondsKarp
{
    struct edge_t { int v, c, r; }; // next destination, capacity, index of reverse node
    vector<vector<edge_t>> gph;
    vector<int> dst, prv, idx;
    EdmondsKarp(int n) : gph(n + 1), dst(n + 1), prv(n + 1), idx(n + 1) {} // '+1' for 1-base nodes
    void add_edge(ints, int e, int c1, int c2 = 0)
    {

```

```

    gph[s].push_back({ e, c1, (int)gph[e].size() });
    gph[e].push_back({ s, c2, (int)gph[s].size() - 1 }); // reverse edge
}

int augment(int s, int t)
{
    fill(dst.begin(), dst.end(), -1);
    queue<int> que;
    que.push(s); dst[s] = 0;
    while (!que.empty())
    {
        int v = que.front(); que.pop();
        for (int i = 0; i < gph[v].size(); i++)
        {
            const auto& e = gph[v][i];
            if (e.c > 0 && dst[e.v] == -1)
            {
                que.push(e.v); dst[e.v] = dst[v] + 1;
                prv[e.v] = v; idx[e.v] = i;
            }
        }
    }
    if (dst[t] == -1) return 0;
    int flow = 1e9;
    for (int i = t; i != s; i = prv[i])
    {
        const auto& e = gph[prv[i]][idx[i]];
        flow = min(flow, e.c);
    }
    for (int i = t; i != s; i = prv[i])
    {
        auto& e = gph[prv[i]][idx[i]];
        e.c -= flow;
        gph[e.v][e.r].c += flow;
    }
    return flow;
}

int run(int s, int t)
{
    int flow = 0, path = 0;
    while (path = augment(s, t)) flow += path;
    return flow;
}
};

```

```

/*
Dinic
*/
struct Dinic
{
    struct edge_t { int v, c, r; };
    int n;
    vector<vector<edge_t>> g;
    vector<int> lv, idx;
    Dinic(int n) :n(n), g(n + 1), lv(n + 1), idx(n + 1) {}
    void add_edge(int s, int e, int c1, int c2 = 0)
    {
        g[s].push_back({ e, c1, (int)g[e].size() });
        g[e].push_back({ s, c2, (int)g[s].size() - 1 });
    }
}

```

```

bool bfs(int s, int t)
{
    fill(lv.begin(), lv.end(), 0);
    queue<int> que;
    que.push(s); lv[s] = 1;
    while (!que.empty())
    {
        int v = que.front(); que.pop();
        for (const auto& e : g[v])
            if (!lv[e.v] && e.c != 0) que.push(e.v), lv[e.v] = lv[v] + 1;
    }
    return lv[t];
}

int dfs(int v, int t, int f = 1e9)
{
    if (v == t || f == 0) return f;
    for (int& i = idx[v]; i < g[v].size(); i++)
    {
        auto& e = g[v][i];
        if (lv[e.v] != lv[v] + 1 || e.c == 0) continue;
        int now = dfs(e.v, t, min(f, e.c));
        if (now == 0) continue;
        e.c -= now;
        g[e.v][e.r].c += now;
        return now;
    }
    return 0;
}

int run(int s, int t)
{
    int flow = 0, path = 0;
    while (bfs(s, t))
    {
        fill(idx.begin(), idx.end(), 0);
        while (path = dfs(s, t)) flow += path;
    }
    return flow;
}

};

/*
Minimum Cost Maximum Flow
*/

struct MCMF
{
    const int INF = 1e9;
    struct edge_t { int v, r, c, d; }; // next destination, index of reverse node, capacity, cost
    vector<vector<edge_t>> g;
    vector<int> in, prv, idx;
    vector<int> dst;
    MCMF(int n) : g(n), in(n), prv(n), idx(n), dst(n) {}
    void add_edge(int s, int e, int c, int d)
    {
        g[s].push_back({ e, (int)g[e].size(), c, d });
        g[e].push_back({ s, (int)g[s].size() - 1, 0, -d });
    }
    bool augment(int s, int t)
    {
        fill(in.begin(), in.end(), 0);

```

```

fill(dst.begin(), dst.end(), INF);
queue<int> que;
que.push(s); dst[s] = 0; in[s] = 1;
while (!que.empty())
{
    int v = que.front(); que.pop(); in[v] = 0;
    for (int i = 0; i < g[v].size(); i++)
    {
        const auto& e = g[v][i];
        if (e.c != 0 && dst[e.v] > dst[v] + e.d)
        {
            dst[e.v] = dst[v] + e.d;
            prv[e.v] = v; idx[e.v] = i;
            if (!in[e.v]) in[e.v] = 1, que.push(e.v);
        }
    }
}
return dst[t] < INF;
}
pair<int, int> minimum_cost_flow(int s, int t)
{
    int flow = 0, cost = 0;
    while (augment(s, t))
    {
        int path = INF;
        for (int i = t; i != s; i = prv[i]) path = min(path, g[prv[i]][idx[i]].c);
        flow += path; cost += path * dst[t];
        for (int i = t; i != s; i = prv[i])
        {
            auto& e = g[prv[i]][idx[i]];
            e.c -= path; g[i][e.r].c += path;
        }
    }
    return { flow, cost };
}
};

```

```

/*
Centroid Decomposition
*/

```

```

struct centroid
{
    int N;
    vector<int> v, sz;
    vector<vector<pair<int, int>>> g;
    centroid(int N) : N(N), v(N + 1), sz(N + 1), g(N + 1) {}
    int get_sz(int x, int p = -1)
    {
        sz[x] = 1;
        for (auto [y, w] : g[x])
            if (y != p && !v[y]) sz[x] += get_sz(y, x);
        return sz[x];
    }
    int get_cent(int x, int p, int SZ)
    {
        for (auto [y, w] : g[x])
            if (y != p && !v[y] && sz[y] * 2 > SZ) return get_cent(y, x, SZ);
        return x;
    }
}

```

```

void dnc(int x)
{
    int cent = get_cent(x, -1, get_sz(x));
    // do something
    v[cent] = 1;
    for (auto [y, w] : g[cent])
        if (!v[y]) dnc(y);
    return;
}
};

/*
Seive of Eratosthenes
*/
struct Eratosthenes
{
    ll N;
    vector<bool> notP;
    Eratosthenes(ll N) : N(N), notP(N + 1, 0) {}
    void go()
    {
        for (ll i = 2; i <= N; i++)
            for (ll j = i * i; j <= N; j += i) // be careful about integer-overflow
                notP[j] = 1;
    }
};

/*
Euler Phi
*/
ll phi(ll n)
{
    ll ret = n;
    for (ll p = 2; p * p <= n; p++)
    {
        if (n % p) continue;
        while (n % p == 0) n /= p;
        ret = ret * (p - 1) / p;
    }
    if (n > 1) ret = ret * (n - 1) / n; // n is prime
    return ret;
}

/*
Modular Inverse
*/
#define MOD 998244353
struct ModInv
{
    ll pow(ll a, ll x)
    {
        if (x == 0) return 1ll;
        ll ret = pow(a, x / 2);
        ret = ret * ret % MOD;
        if (x % 2) ret = ret * a % MOD;
        return ret;
    }
    ll FLT(ll a) { return pow(a, MOD - 2); } // MOD should be prime number
};

```

```

/*
Extended Euclid
*/

/*
Chinese Remainder Theorem
*/
int chineseRemainderTheorem(const std::vector<int>& num, const std::vector<int>& rem) {
    int k = num.size();
    int prod = 1;
    for (int i = 0; i < k; i++) {
        prod *= num[i];
    }

    int result = 0;

    for (int i = 0; i < k; i++) {
        int pp = prod / num[i];
        result += rem[i] * modInverse(pp, num[i]) * pp;
    }

    return result % prod;
}

```

```

/*
Catalan Number
*/
vector<ll> Catalan(ll n)
{
    vector<ll> c(n + 1);
    c[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j < i; j++) c[i] += c[j] * c[i - 1 - j];
    return c;
}

```

```

/*
Binomial Coefficient ( $N^2$ )
*/
vector<vector<ll>> Combination(int n)
{
    vector<vector<ll>> c(n + 1, vector<ll>(n + 1));
    for (int i = 0; i <= n; i++)
    {
        c[i][0] = 1;
        for (int j = 1; j <= i; j++) c[i][j] = c[i - 1][j - 1] + c[i - 1][j];
    }
    return c;
}

```

```

/*
Lucas Theorm (Binomial Coefficient)
*/
struct Lucas
{
    //init:  $O(P + \log P)$ , query:  $O(\log P)$ 
    const size_t P;
}

```

```

vector<ll> fac, inv;
ll Pow(ll a, ll b)
{
    ll ret = 1;
    for (; b; b >>= 1, a = a * a % P) if (b & 1) ret = ret * a % P;
    return ret;
}
Lucas(size_t P) : P(P), fac(P), inv(P)
{
    fac[0] = 1;
    for (int i = 1; i < P; i++) fac[i] = fac[i - 1] * i % P;
    inv[P - 1] = Pow(fac[P - 1], P - 2);
    for (int i = P - 2; ~i; i--) inv[i] = inv[i + 1] * (i + 1) % P;
}
ll small(ll n, ll r) const
{
    if (n < r) return 0;
    return fac[n] * inv[r] % P * inv[n - r] % P;
}
ll calc(ll n, ll r) const
{
    if (n < r || n < 0 || r < 0) return 0;
    if (!n || !r || n == r) return 1;
    return small(n % P, r % P) * calc(n / P, r / P) % P;
}
};

/*
Rabin - Karp
*/
template<ll P, ll M> struct Hashing
{
    vector<ll> H, B;
    void build(const string& S)
    {
        H.resize(S.size() + 1);
        B.resize(S.size() + 1);
        B[0] = 1;
        for (int i = 1; i <= S.size(); i++) H[i] = (H[i - 1] * P + S[i - 1]) % M;
        for (int i = 1; i <= S.size(); i++) B[i] = B[i - 1] * P % M;
        // H[i] := hash(S[0]...S[i-1])
    }
    ll get(int s, int e) // get(s, e) := hash(S[s-1]...S[e-1])
    {
        ll res = (H[e] - H[s - 1] * B[e - s + 1]) % M;
        return res >= 0 ? res : res + M;
    }
};

/*
Trie
*/
struct Trie
{
    Trie* ch[26];
    int key;
    Trie()
    {
        fill(ch, ch + 26, nullptr);
    }
};

```

```

        key = -1;
    }
    ~Trie()
    {
        for (int i = 0; i < 26; i++) delete ch[i];
    }
    void insert(const char* s, int id)
    {
        if (*s == 0) { key = id; return; }
        if (!ch[*s - 'a']) ch[*s - 'a'] = new Trie();
        ch[*s - 'a']->insert(s + 1, id);
    }
    int find(const char* s)
    {
        if (*s == 0) return key;
        if (!ch[*s - 'a']) return -1;
        else return ch[*s - 'a']->find(s + 1);
    }
};

/*
KMP
*/
vector<int> GetFail(const string& p)
{
    int m = p.size();
    vector<int> fail(m);
    fail[0] = 0;
    for (int i = 1, j = 0; i < m; i++)
    {
        while (j > 0 && p[i] != p[j]) j = fail[j - 1];
        if (p[i] == p[j]) fail[i] = ++j;
    }
    return fail;
}

vector<int> KMP(const string& s, const string& p)
{
    int n = s.size(), m = p.size();
    vector<int> fail = GetFail(p), res;
    for (int i = 0, j = 0; i < n; i++)
    {
        while (j > 0 && s[i] != p[j]) j = fail[j - 1];
        if (s[i] == p[j])
        {
            if (j + 1 == m) res.push_back(i - m + 1), j = fail[j];
            else j++;
        }
    }
    return res;
}

/*
Manacher
*/
struct manacher
{
    string s; // input string
    string ss = ""; // preprocessed string
    vector<ll> ma; // ma[i] := length of palindrome which center is i

```



```

    ll r = -1, c = -1; // center of previous palindrome, radius of previous palindrome
    manacher(string& s) : s(s), ma(s.size() * 2 + 2, 0) {}

    void go()
    {
        for (char cc : s)
        {
            ss.push_back('#'); // dummy character for even length palindrome
            ss.push_back(cc);
        }
        ss.push_back('#');
        for (ll i = 0; i < ss.size(); i++)
        {
            if (i <= r) ma[i] = min(r - i, ma[2 * c - i]); // get ma[i]
            while (i + ma[i] + 1 < ss.size() && i - ma[i] - 1 >= 0 && ss[i + ma[i] + 1] == ss[i - ma[i] - 1])
                ma[i]++; // update ma[i]
            if (i + ma[i] > r) // update c and r
            {
                c = i;
                r = i + ma[i];
            }
        }
    }

    // real index of palindrome(0-base): [(i-ma[i]+1)/2, (i+ma[i]-1)/2]
};

/*
Suffix Array
*/

struct SuffixArray
{
    int n;
    string s;
    vector<int> sfx, g, ng, cnt, idx; // sfx[i] := s에서 사전 순 i번째 접미사의 첫 글자 위치
    vector<int> rev, lcp; // lcp[i] := suffix array 상에서 i번째 접미사와 i-1번째 접미사의 Longest Common Prefix
    SuffixArray(string& str)
    {
        s = str;
        n = str.length();
        sfx.resize(n + 1);
        g.resize(2 * n + 1);
        ng.resize(2 * n + 1);
        cnt.resize(n + 1);
        idx.resize(n + 1);

        rev.resize(n + 1);
        lcp.resize(n + 1);
    }

    void getsfx()
    {
        s = ' ' + s;
        vector<int> tmp;
        for (int i = 1; i <= n; i++)
        {
            sfx[i] = i; // neccessary?
            tmp.push_back(s[i]);
        }
        sort(tmp.begin(), tmp.end());
        tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
        for (int i = 1; i <= n; i++)

```

```

        g[i] = lower_bound(tmp.begin(), tmp.end(), s[i]) - tmp.begin() + 1;
    for (int i = n + 1; i <= 2 * n; i++) g[i] = 0;
    for (int t = 1; t < n; t *= 2)
    {
        // counting sort by g[i+t] for stable sorting
        for (int i = 0; i <= n; i++) cnt[i] = 0;
        for (int i = 1; i <= n; i++) cnt[g[i + t]]++;
        for (int i = 1; i <= n; i++) cnt[i] += cnt[i - 1];
        for (int i = n; i >= 1; i--) idx[cnt[g[i + t]]--] = i;
        // counting sort by g[i] for stable sorting
        for (int i = 0; i <= n; i++) cnt[i] = 0;
        for (int i = 1; i <= n; i++) cnt[g[i]]++;
        for (int i = 1; i <= n; i++) cnt[i] += cnt[i - 1];
        for (int i = n; i >= 1; i--) sfx[cnt[g[idx[i]]]--] = idx[i];
        // prepare for next update
        ng[sfx[1]] = 1;
        for (int i = 2; i <= n; i++)
            ng[sfx[i]] = ng[sfx[i - 1]] + (g[sfx[i - 1]] < g[sfx[i]] || (g[sfx[i - 1]] == g[sfx[i]] && g[sfx[i - 1]] +
t] < g[sfx[i] + t)));
        for (int i = 1; i <= n; i++) g[i] = ng[i];
    }
}

void getlcp()
{
    for (int i = 1; i <= n; i++) rev[sfx[i]] = i;
    int now = 0;
    for (int i = 1; i <= n; i++)
    {
        if (rev[i] - 1 >= 1)
        {
            while (s[i + now] == s[sfx[rev[i] - 1] + now]) now++;
            lcp[rev[i]] = now;
        }
        if (now) now--;
    }
}
};

/*
Union Find
*/
struct UF
{
    int n;
    vector<int> p;
    UF(int n) : n(n), p(n + 1, -1) {}
    int find(int x)
    {
        if (p[x] < 0) return x;
        return p[x] = find(p[x]);
    }
    void merge(int a, int b)
    {
        a = find(a); b = find(b);
        if (a == b) return;
        p[b] = a;
    }
};

```

```

/*
Ternary Search
*/
double TernarySearch(double l, double r)
{
    while (l + 1e-9 <= r)
    {
        double m1 = (2 * l + r) / 3;
        double m2 = (l + 2 * r) / 3;
        if (f(m1) > f(m2)) l = m1; // find min
        else r = m2;
    }
    return l;
}

ll TernarySearch(ll l, ll r)
{
    while (l + 3 <= r)
    {
        ll m1 = (2 * l + r) / 3;
        ll m2 = (l + 2 * r) / 3;
        if (f(m1) > f(m2)) l = m1; // find min
        else r = m2;
    }
    ll ret = INF;
    for (int i = l; i <= r; i++) ret = min(ret, f(i));
    return ret;
}

```

```

/*
Power with Divide and Conquer
*/

```

```

ll pow(ll a, ll x)
{
    if (x == 0) return 1ll;
    ll ret = pow(a, x / 2);
    ret = ret * ret % MOD;
    if (x % 2) ret = ret * a % MOD;
    return ret;
}

```

```

/*
Fibonacci Matrix
*/

```

```

struct FiboMat
{
    struct matrix22 { ll f[2][2]; };
    matrix22 mul(matrix22 a, matrix22 b)
    {
        matrix22 c;
        c.f[0][0] = a.f[0][0] * b.f[0][0] + a.f[0][1] * b.f[1][0];
        c.f[0][1] = a.f[0][0] * b.f[0][1] + a.f[0][1] * b.f[1][1];
        c.f[1][0] = a.f[1][0] * b.f[0][0] + a.f[1][1] * b.f[1][0];
        c.f[1][1] = a.f[1][0] * b.f[0][1] + a.f[1][1] * b.f[1][1];
        return c;
    }
    matrix22 pow(matrix22 a, int n)
    {
        if (n > 1)
        {

```

```

        a = pow(a, n / 2);
        a = mul(a, a);
        if (n % 2 == 1)
        {
            matrix22 f1 = { 1, 1, 1, 0 };
            a = mul(a, f1);
        }
    }
    return a;
}

ll fibo(int n)
{
    matrix22 a = { 1, 1, 1, 0 };
    a = pow(a, n);
    return a.f[0][1];
}

};

/*
Convex Hull Trick
*/

struct CHT
{
    struct LinearFunc // f(x)=ax+b, x>=s
    {
        ll a, b;
        double s; // starting point of segment
        LinearFunc() : LinearFunc(1, 0) {}
        LinearFunc(ll a1, ll b1) : a(a1), b(b1), s(0) {}
    };
    // return x coordinate of intersection of line f and g
    double cross(LinearFunc& f, LinearFunc& g) { return (g.b - f.b) / (f.a - g.a); }
    int N, top = 0;
    vector<LinearFunc> f;
    CHT(int N) : N(N), f(N) {}
    void addLine(LinearFunc g)
    {
        while (top > 0)
        {
            g.s = cross(f[top - 1], g);
            if (g.s > f[top - 1].s) break;
            top--; // pop
        }
        f[top++] = g; // add
    }
    int find(int x) // find segment which includes x coordinate
    {
        int fpos = top - 1;
        if (x < f[top - 1].s)
        {
            int l = 0, r = top - 1;
            while (l <= r)
            {
                int mid = (l + r) / 2;
                if (x >= f[mid].s)
                {
                    l = mid + 1;
                    fpos = mid;
                }
            }
        }
    }

```

```

        else r = mid - 1;
    }
}
return fpos;
}
};

//-----

// 6.3 Bit Twiddling Hack
int __builtin_clz(int x); // number of leading zero
int __builtin_ctz(int x); // number of trailing zero
int __builtin_clzll(long long x); // number of leading zero
int __builtin_ctzll(long long x); // number of trailing zero
int __builtin_popcount(int x); // number of 1-bits in x
int __builtin_popcountll(long long x); // number of 1-bits in x
lsb(n): (n & -n); // last bit (smallest)
floor(log2(n)): 31 - __builtin_clz(n | 1);
floor(log2(n)): 63 - __builtin_clzll(n | 1);

// compute next perm. ex) 00111, 01011, 01101, 01110, 10011, 10101..
long long next_perm(long long v){
    long long t = v | (v-1);
    return (t + 1) | (((~t & ~t) - 1) >> (__builtin_ctz(v) + 1));
}

// vector resizing
vector<vector<int>>> vec;
vec.resize(rows, vector<int>(cols, 0));

```

1. Data Structure (자료구조)

- **Segment Tree (세그먼트 트리):** 배열의 구간 합, 최솟값 등의 구간 쿼리를 빠르게 처리하고 업데이트할 때 사용합니다.
- **Lazy Propagation (레이지 프로파게이션):** 세그먼트 트리에서 특정 구간 전체의 값을 한 번에 효율적으로 업데이트할 때 사용합니다.
- **Kosaraju (SCC, 코사라주):** 방향 그래프에서 서로 긴밀하게 연결된 부분 그래프인 강한 결합 요소(SCC)를 찾을 때 사용합니다.
- **LCA (최소 공통 조상):** 트리 구조에서 두 노드의 가장 가까운 공통 조상을 빠르게 찾을 때 사용합니다.
- **Merge Sort Tree (머지 소트 트리):** 특정 구간 내에서 주어진 값보다 작거나 같은 원소의 개수를 찾는 등의 구간 정렬 쿼리에 사용합니다.
- **Fenwick Tree (펜윅 트리 / BIT):** 배열의 특정 인덱스 값 업데이트와 누적 구간 합을 $O(\log N)$ 시간에 빠르고 메모리 효율적으로 처리하는 이진 인덱스 트리 자료구조입니다.
- **Sparse Table (스파스 테이블):** 값이 변하지 않는 배열에서 구간 최솟값/최댓값(RMQ) 쿼리를 $O(1)$ 의 매우 빠른 시간에 처리할 때 사용합니다.

2. Geometry (기하학)

- **CCW:** 세 점의 방향성(반시계, 시계, 일직선)을 판별하기 위해 외적을 활용할 때 사용합니다.
- **360 degree sort (각도 정렬):** 특정 기준점 주변의 점들을 반시계 방향 등의 각도 순서대로 정렬할 때 사용합니다.
- **Convex Hull (볼록 껍질):** 2차원 평면의 점들을 모두 포함하는 가장 작은 볼록 다각형을 구성할 때 사용합니다.
- **Rotating Calipers (회전하는 캘리퍼스):** 볼록 껍질 위에서 가장 멀리 떨어진 두 점 사이의 최대 거리를 구할 때 사용합니다.
- **Point in Convex Polygon (볼록 다각형 내부 점 판별):** 특정 점이 주어진 볼록 다각형 내부에 위치하는지 이분 탐색을 통해 판별할 때 사용합니다.

3. Graph (그래프)

- **Bellman-Ford (벨만-포드):** 음수 가중치 간선이 있는 그래프에서 최단 경로를 구하거나 음수 사이클을 판별할 때 사용합니다.
- **Eulerian Circuit (오일러 회로):** 그래프의 모든 간선을 정확히 한 번씩 지나는 경로(회로)를 찾을 때 사용합니다.
- **Bipartite Matching (이분 매칭):** 이분 그래프에서 두 집합 간의 최대 매칭 수를 구할 때 사용합니다.

- **Edmonds-Karp (에드몬드-카프):** 네트워크 플로우에서 최대 유량을 구하기 위해 BFS를 활용하는 알고리즘입니다.
- **Dinic (디닉):** 에드몬드-카프보다 더 빠른 속도로 네트워크의 최대 유량을 계산할 때 사용합니다.
- **Minimum Cost Maximum Flow (MCMF):** 네트워크에서 최대 유량을 흘리면서 동시에 최소의 비용이 드는 경로를 찾을 때 사용합니다.
- **Centroid Decomposition (센트로이드 분할):** 트리를 센트로이드(무게 중심) 기준으로 분할하여 트리 상의 경로 문제를 효율적으로 풀 때 사용합니다.

4. Math (수학)

- **Seive of Eratosthenes (에라토스테네스의 체):** 특정 범위 내의 모든 소수를 빠르게 찾아낼 때 사용합니다.
- **Euler Phi (오일러 피 함수):** 자연수 n 과 서로소인 1부터 n 까지의 자연수 개수를 구할 때 사용합니다.
- **Modular Inverse (모듈러 역원):** 모듈러 연산에서 나눗셈을 수행하기 위해 거듭제곱을 이용해 역원을 구할 때 사용합니다.
- **Extended Euclid (확장 유클리드):** 두 수의 최대공약수와 베주 항등식을 만족하는 정수 해를 구할 때 사용합니다.
- **Chinese Remainder Theorem (중국인의 나머지 정리):** 서로소인 모듈러들에 대한 연립 합동 방정식의 해를 찾을 때 사용합니다.
- **Catalan Number (카탈란 수):** 올바른 괄호 문자열, 이진 트리 등 조합론적인 경우의 수를 계산할 때 사용합니다.
- **Binomial Coefficient (이항 계수):** 파스칼의 삼각형 원리를 이용해 N 의 크기가 작을 때 조합(nCr)을 구할 때 사용합니다.
- **Lucas Theorm (루카스 정리):** 매우 큰 수의 이항 계수를 작은 소수로 나눈 나머지를 구할 때 사용합니다.

5. String (문자열)

- **Rabin-Karp (라빈-카프):** 해시를 활용하여 긴 문자열 내에서 특정 패턴 문자열을 빠르게 검색할 때 사용합니다.
- **Trie (트라이):** 다수의 문자열 집합에서 특정 문자열의 삽입과 탐색을 빠르게 처리하기 위한 트리 자료구조입니다.
- **KMP:** 접두사와 접미사의 일치 길이(Fail 함수)를 활용해 텍스트 내에서 패턴을 빠르게 찾을 때 사용합니다.
- **Manacher (마나처):** 문자열 내에서 존재하는 가장 긴 팰린드롬(회문) 부분 문자열을 선형 시간에 찾을 때 사용합니다.
- **Suffix Array (접미사 배열):** 문자열의 모든 접미사를 사전순으로 정렬하여 최장 공통 접두사(LCP) 등 복잡한 문자열 문제에 사용합니다.

6. ETC (기타 알고리즘)

- **Union Find (유니온 파인드):** 여러 개의 노드가 서로 같은 집합에 속해 있는지 판별하고 병합하는 분리 집합 구조에 사용합니다.
- **Ternary Search (삼분 탐색):** 볼록하거나 오목한 유니모달(단봉) 함수에서 극값을 효율적으로 찾을 때 사용합니다.
- **Power with Divide and Conquer (분할 정복을 이용한 거듭제곱):** 지수가 매우 큰 거듭제곱 연산을 분할 정복을 통해 $O(\log N)$ 으로 빠르게 계산할 때 사용합니다.
- **Fibonacci Matrix (피보나치 행렬):** 행렬의 거듭제곱을 이용하여 피보나치 수열의 N 번째 항을 빠르게 구할 때 사용합니다.
- **Convex Hull Trick (볼록 껍질 최적화):** 1차 함수들의 최솟값 또는 최댓값을 구하는 형태의 특정 DP(동적 계획법) 점화식을 최적화할 때 사용합니다.

🔥 제출 전 확인 / Common Errors

1. 자료형 및 오버플로우 (Data Types)

- [] **Integer Overflow:** 계산 중간 과정이나 최종 결과가 int 범위(약 2×10^9)를 초과하지 않는가? (필요시 모두 long long으로 변경)
- [] **1LL 캐스팅:** long long = int * int 형태에서 오버플로우가 나지 않는가? (1LL * int * int로 수정)
- [] **부동소수점 오차:** double 비교 시 == 대신 epsilon($1e-9$ 등)을 더하거나 빼서 비교했는가? (가급적 정수 연산 사용)

2. 배열 및 메모리 (Arrays & Bounds)

- [] **Out of Bounds (OOB):** 배열 크기를 N 대신 $N+10$ 정도로 넉넉하게 선언했는가?
- [] **Off-by-One Error:** 0-indexed와 1-indexed를 혼용해서 발생한 인덱스 에러는 없는가?
- [] **음수 인덱스 접근:** 배열 인덱스가 0 미만으로 내려가는 경우가 없는가?

3. 다중 테스트 케이스 (Multi Test Cases - 빈출 1순위)

- [] **초기화:** 전역 변수, vector, 배열(memset, fill)을 매 테스트 케이스마다 완전히 초기화했는가?

- [] **Break/Continue 오작동:** 루프 중간에 탈출할 때, 미처 입력받지 못한 나머지 입력값들이 다음 테스트 케이스에 영향을 주지 않는가?

- [] **출력 형식:** Case #1: 형식, 공백, 줄바꿈 등을 요구사항에 맞게 출력했는가?

4. 연산 및 로직 (Math & Logic)

- [] **Division by Zero:** 0으로 나누거나 모듈러(% 0) 연산을 하는 경우가 없는가?
- [] **음수 모듈러 연산:** 뺄셈에 대한 모듈러 연산 시 음수가 나오지 않는가? $((A - B \% MOD + MOD) \% MOD)$
- [] **비트 연산자 우선순위:** &, |, ^ 연산자는 비교 연산자(==)보다 우선순위가 낮으므로 괄호를 적절히 씌웠는가? (if ((x & 1) == 0))

Edge Cases (극단적 예외 상황)

- [] **최솟값 입력:** $N = 0$ 또는 $N = 1$ 일 때 코드가 정상적으로 작동하는가? (예: $N=1$ 일 때 트리 간선이 없는 경우)
- [] **최댓값 입력:** N 이 문제에서 주어진 최댓값일 때 TLE(시간 초과)나 MLE(메모리 초과)가 발생하지 않는가?
- [] **그래프 특이 케이스:**
 - 그래프가 여러 개로 단절되어 있는 경우 (Disconnected Graph)
 - 두 정점 사이에 간선이 여러 개인 경우 (Multiple Edges)
 - 자기 자신으로 돌아오는 간선이 있는 경우 (Self Loop)
- [] **기하 특이 케이스:**
 - 세 점이 일직선 상에 있는 경우 (Collinear)
 - 두 점의 좌표가 완전히 동일한 경우
- [] **문자열 특이 케이스:** 빈 문자열이 주어지거나, 길이가 1인 문자열인 경우

"가장 어두운 밤에도 별은 빛난다." - 랄프 왈도 에머슨

"완벽을 두려워하지 마라. 어차피 완벽에 도달할 수는 없으니까." - 살바도르 달리

"인생에서 가장 큰 영광은 넘어지지 않는 데 있는 것이 아니라, 넘어질 때마다 일어나는 데 있다." - 넬슨 만델라

성공한 사람과 그렇지 못한 사람의 가장 큰 차이는 성공한 사람은 자신의 목표를 정확히 알고 그 목표를 이루기 위해 매일 노력한다는 점이다." - 데니스 웨이틀리. 성공은 우연히 얻어지는 것이 아닙니다. 명확한 목표 설정과 꾸준한 노력, 이 두 가지가 성공의 열쇠입니다. SMART 기법을 활용하여 구체적이고 측정 가능하며, 달성 가능하고, 관련성 있고, 시간 제약이 있는 목표를 설정해 보세요.

당신의 마음은 정원과 같습니다. 당신이 원하는 대로 꽃을 심고 가꿀 수 있습니다." 마음 챙김과 명상은 마음 정원을 가꾸는 효과적인 도구입니다. 하루 10분, 고요한 시간 속에서 자신의 내면에 집중하며 마음의 평화를 되찾아보세요. MBSR(마음 챙김에 기반한 스트레스 완화) 프로그램, 마음 챙김 앱 등 다양한 방법을 활용할 수 있습니다.



